

TP Symfony : Création d'une application web

Table des matières

1) Problématique.....	1
2) Prérequis	1
3) Initialisation et configuration	1
4) Exercices	3
5) Résultats :.....	7
6) Conclusion.....	8

1) Problématique

Nous souhaitons créer une application web de gestion d'évènements, pour cela nous utiliserons le framework PHP Symfony, l'outil Composer, PHP 8.2+ ainsi que XAMPP pour la partie MySQL.

2) Prérequis

Les outils permettant de répondre à la problématique peuvent se trouver ici :

Symfony : <https://symfony.com/download>

Composer : <https://getcomposer.org/download/>

XAMPP : <https://www.apachefriends.org/>

PHP : **scoop install php** sur cmd

3) Initialisation et configuration

Nous allons commencer par créer un projet web complet. Pour cela on crée d'abord un dossier pour le projet puis on le sélectionne sur cmd avec **cd**.

Quand on est dessus on effectue les deux commandes suivantes :

```
Microsoft Windows [version 10.0.26200.9278]
(c) Microsoft Corporation. Tous droits réservés.

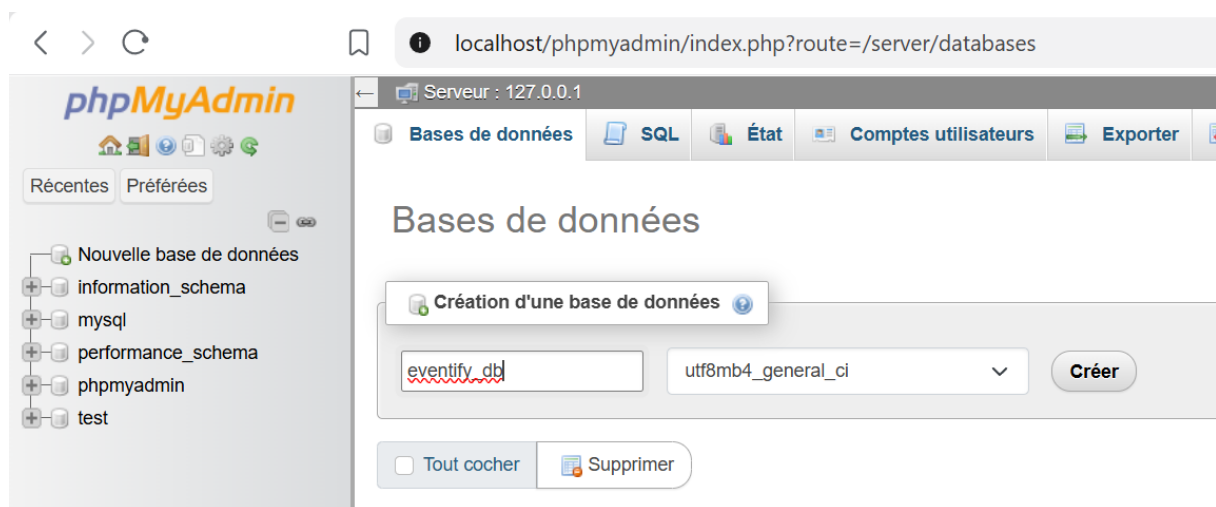
C:\Users\Ewan Alexandre\Desktop\TP Symfony>symfony new eventify --webapp
* Creating a new Symfony project with Composer
* Setting up the project under Git version control
  (running git init C:\Users\Ewan Alexandre\Desktop\TP Symfony\eventify)

[OK] Your project is now ready in C:\Users\Ewan Alexandre\Desktop\TP Symfony\eventify

C:\Users\Ewan Alexandre\Desktop\TP Symfony>cd eventify
C:\Users\Ewan Alexandre\Desktop\TP Symfony\eventify>
```

Nous allons ensuite configurer la base de données grâce à XAMPP. Il faut activer Apache et MySQL puis accéder à phpmyadmin.

Dans phpmyadmin on crée manuellement la base de données :



Ensuite dans l'explorateur de fichiers on accède au dossier eventify créé après la commande faite précédemment. Et dans ce dossier il faut trouver le fichier .env , le modifier avec bloc-notes, chercher « DATABASE_URL= » avec CTRL+F et remplacer par :

DATABASE_URL="mysql://root:@127.0.0.1:3306/eventify_db?serverVersion=8.0&characterSet=utf8mb4"

(il faut remplacer la ligne « DATABASE_URL » où il n'y a pas de #)

Ensuite on utilise la commande suivante pour vérifier que la connexion marche :

```
C:\Users\Ewan Alexandre\Desktop\TP Symfony\eventify>php bin/console doctrine:database:create --if-not-exists
Database 'eventify_db' for connection named default already exists. Skipped.

C:\Users\Ewan Alexandre\Desktop\TP Symfony\eventify>
```

Comme on peut le voir tout fonctionne.

4) Exercices

1) Création de l'entité event

Nous devons tout d'abord générer une entité event. L'entité event est facile à créer car il suffit simplement de préciser chaque détails par rapport aux questions posées :

```
C:\Users\Ewan Alexandre\Desktop\TP Symfony\eventify>php bin/console make:entity Event
created: src/Entity/Event.php
created: src/Repository/EventRepository.php

Entity generated! Now let's add some fields!
You can always add more fields later manually or by re-running this command.

New property name (press <return> to stop adding fields):
> title

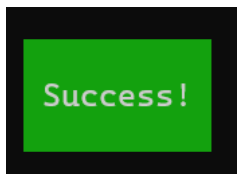
Field type (enter ? to see all types) [string]:
> string

Field length [255]:
> 255

Can this field be null in the database (nullable) (yes/no) [no]:
> no
```

Ensuite il suffit de faire ça avec l'ensemble des entités que l'on souhaite créer.

Lorsqu'on a fini il suffit simplement de cliquer sur Entrer pour confirmer la fin :



2) Migration

On peut migrer les données pour créer la table dans MySQL de cette manière :

```
Next: When you're ready, create a migration with php bin/console make:migration

C:\Users\Ewan Alexandre\Desktop\TP Symfony\eventify>php bin/console make:migration
created: migrations/Version20260904072516.php
```



On peut ensuite faire une migration de la doctrine pour indiquer une modification à faire dans la base de données de cette manière (il faudra confirmer avec « yes ») :

```
C:\Users\Ewan Alexandre\Desktop\TP Symfony\eventify>php bin/console doctrine:migrations:migrate

WARNING! You are about to execute a migration in database "eventify_db" that could result in schema changes and data loss. Are you sure you wish to continue? (yes/no) [yes]:
> yes

[notice] Migrating up to DoctrineMigrations\Version20260904072516
[notice] finished in 187.4ms, used 22M memory, 1 migrations executed, 2 sql queries

[OK] Successfully migrated to version: DoctrineMigrations\Version20260904072516
```

3) Backend et CRUD

On va pouvoir générer notre contrôleur et les formulaires à l'aide d'une simple commande, comme ceci :

```
C:\Users\Ewan Alexandre\Desktop\TP Symfony\eventify>php bin/console make:crud Event

Choose a name for your controller class (e.g. EventController) [EventController]:
> EventController

Do you want to generate PHPUnit tests? [Experimental] (yes/no) [no]:
> no

created: src/Controller/EventController.php
created: src/Form/EventType.php
created: templates/event/_delete_form.html.twig
created: templates/event/_form.html.twig
created: templates/event/edit.html.twig
created: templates/event/index.html.twig
created: templates/event/new.html.twig
created: templates/event/show.html.twig

Success!
```

Exercice guidé : les règles de validation

Nous allons ajouter des contraintes de validation dans notre Contrôleur, pour cela il faut donc accéder au fichier php présent dans :

```
TP Symfony > eventify > src > Entity
```

Nous allons ouvrir avec Notepad++ (mais le mieux reste VSCode).

Dans le code, on ajoute tout d'abord (grâce à l'aide qui nous est donné) un outil de cette façon :

```
use Symfony\Component\Validator\Constraints as Assert;
```

Puis on rajoute une contrainte pour le titre avec les Assert :

```
#[ORM\Column(length: 255)]
#[Assert\NotBlank(message: "Le titre ne peut pas être vide.")]
#[Assert\Length(min: 5, minMessage: "Le titre doit faire au moins {{ limit }} caractères.")]
private ?string $title = null;
```

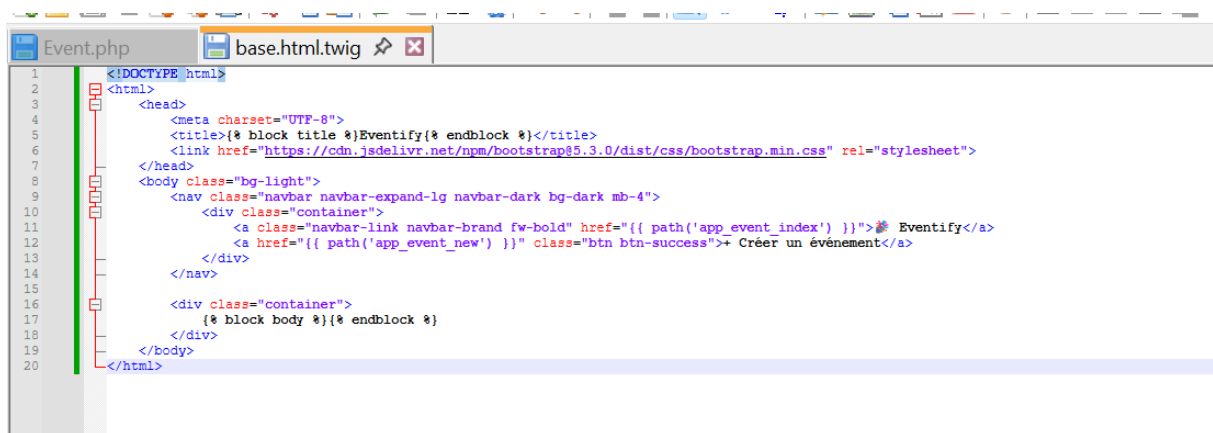
Et on fait pareil pour la capacité :

```
#[ORM\Column]
#[Assert\Positive(message: "La capacité doit être un nombre positif.")]
private ?int $capacity = null;
```

Exercice guidé : Layout de base

Nous allons cette fois-ci aller chercher le fichier base.html.twig dans le dossier templates.

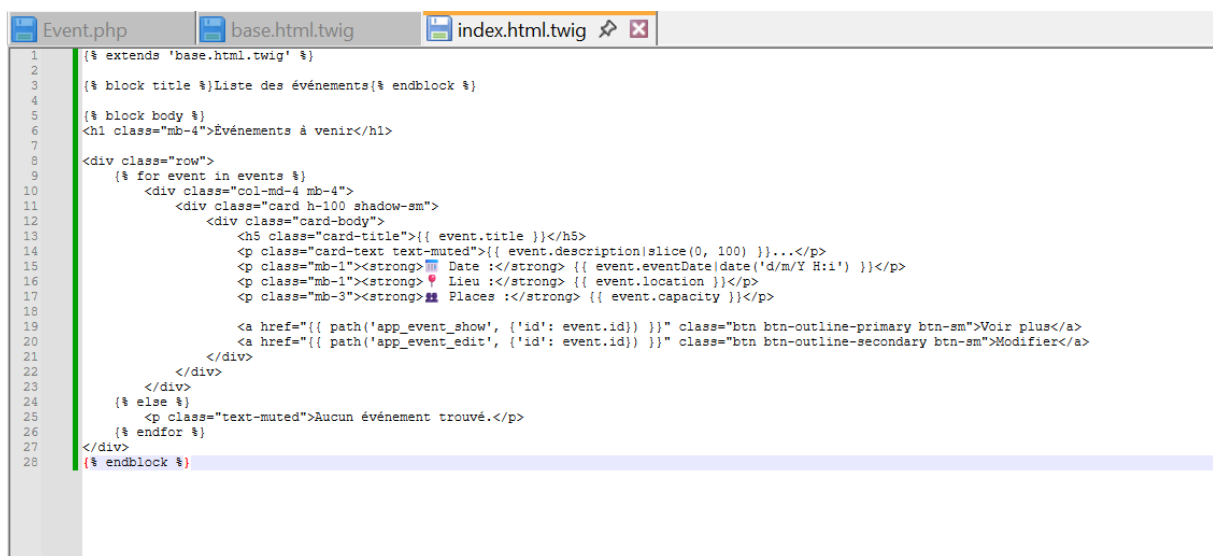
On l'ouvre avec Notepad++ puis on utilise l'aide donné pour remplacer tout le contenu du fichier :



```
1 <!DOCTYPE html>
2 <html>
3 <head>
4 <meta charset="UTF-8">
5 <title>{% block title %}Eventify{% endblock %}</title>
6 <link href="https://cdn.jsdelivr.net/npm/bootstrap@5.3.0/dist/css/bootstrap.min.css" rel="stylesheet">
7 </head>
8 <body class="bg-light">
9 <nav class="navbar navbar-expand-lg navbar-dark bg-dark mb-4">
10 <div class="container">
11 <a class="navbar-link navbar-brand fw-bold" href="{{ path('app_event_index') }}">Eventify</a>
12 <a href="{{ path('app_event_new') }}" class="btn btn-success">Créer un événement</a>
13 </div>
14 </nav>
15
16 <div class="container">
17 <div class="container">
18 <div class="container">
19 <div class="container">
20 </div>
```

Exercice guidé : Reformatage de la liste

Comme pour l'exercice d'avant il va simplement falloir modifier le contenu du fichier index.html.twig présent dans templates\event\index.html.twig :



```
1 {% extends 'base.html.twig' %}
2
3 {% block title %}Liste des événements{% endblock %}
4
5 {% block body %}
6 <h1 class="mb-4">Événements à venir</h1>
7
8 <div class="row">
9 <div class="row">
10 <div class="row">
11 <div class="row">
12 <div class="row">
13 <div class="row">
14 <div class="row">
15 <div class="row">
16 <div class="row">
17 <div class="row">
18 <div class="row">
19 <div class="row">
20 <div class="row">
21 <div class="row">
22 <div class="row">
23 <div class="row">
24 <div class="row">
25 <div class="row">
26 <div class="row">
27 <div class="row">
28 </div>
```

Travail à réaliser :

1. Ajouter un champ de recherche par titre dans la page de liste des événements (Front-End) et 2. Modifier la méthode dans EventRepository.php pour filtrer les résultats selon la saisie.

Pour mettre en place le champ de recherche par titre, nous avons dû relier le front-end et le back-end de notre application :

Tout d'abord, nous avons modifié le fichier index.html.twig dans templates/event/ afin d'ajouter un formulaire de recherche avec un champ texte q au-dessus de la liste des événements.

Ensuite, dans le contrôleur EventController.php, nous avons adapté la fonction index() pour récupérer la saisie de l'utilisateur grâce à `$request->get('q')`. Si une recherche est effectuée, le contrôleur appelle notre méthode personnalisée `champRecherche()` créée dans EventRepository.php, qui filtre les événements avec une requête Doctrine contenant la condition LIKE. S'il n'y a pas de recherche, la page continue d'afficher tous les événements par défaut.

3. Ajouter un bouton « S'inscrire » :

Pour ajouter le bouton, on est simplement allé dans le fichier show.html.twig dans templates/event/. Juste sous le tableau des détails de l'événement, on a rajouté un bouton vert Bootstrap pour s'inscrire. On a mis une petite condition avec un `{% if %}` pour afficher le bouton seulement s'il reste des places, et on affiche dedans le nombre de places restantes à côté du texte.

4. Décrémenter la capacité :

Pour faire marcher le bouton, on a créé une fonction toute simple appelée `inscription()` dans EventController.php. Dès qu'on clique sur le bouton, la fonction récupère l'événement, vérifie qu'il reste au moins 1 place, et fait un simple - 1 sur la capacité. Ensuite on utilise `$entityManager->flush()` pour sauvegarder le nouveau chiffre dans la base de données MySQL, puis on recharge la page de l'événement pour voir le résultat. Tout le code est consultable sur le dépôt Git que j'ai envoyé.

5. Si la capacité atteint 0, afficher un badge « Complet » et désactiver le bouton :

Dans notre condition `{% if %}` du fichier show.html.twig, on a rajouté la partie `{% else %}` pour gérer le cas où la capacité tombe à 0. Quand il n'y a plus de places, on affiche un petit badge rouge avec écrit « Complet » et on met le mot-clé disabled sur le bouton

pour qu'il devienne gris et qu'on ne puisse plus cliquer dessus. Tout le fonctionnement est vérifiable sur le dépôt Git que j'ai envoyé.

6. Personnaliser le rendu du formulaire de création/modification avec les classes Bootstrap :

De base, le formulaire créé par Symfony était tout simple et pas très beau. Du coup, on a ouvert le fichier `_form.html.twig` et on a rajouté les classes de base de Bootstrap : `form-control` pour avoir de belles cases de saisie, et `btn-primary` pour avoir un bouton bleu. On a aussi mis la date et le nombre de places côte à côte sur la même ligne pour que ce soit plus propre visuellement. Le résultat et le code sont disponibles sur le dépôt Git que j'ai envoyé.

5) Résultats :

On va désormais lancer le serveur pour tester avec la commande suivante :

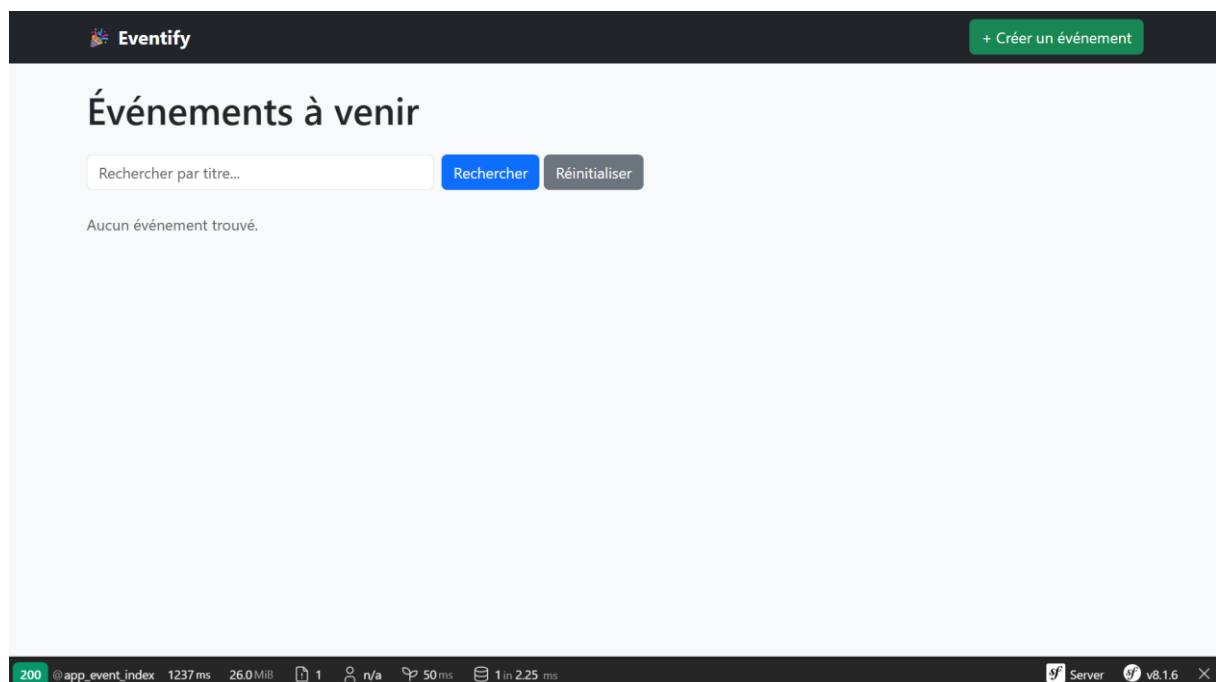
```
C:\Users\Ewan Alexandre\Desktop\TP Symfony\eventify>symfony server:start

[WARNING] run "symfony.exe server:ca:install" first if you want to run the web server with TLS support, or use
"--p12" or "--no-tls" to avoid this warning

Following Web Server log file (C:\Users\Ewan Alexandre\AppData\Roaming\symfony-cli\log\39e7e2ac472356b013b89202ac7c4600d087d74b.log)
Following PHP-CGI log file (C:\Users\Ewan Alexandre\AppData\Roaming\symfony-cli\log\39e7e2ac472356b013b89202ac7c4600d087d74b\79ca75f9e90b4126a5955a33ea6a41ec5e854698.log)

[WARNING] The local web server is optimized for local development and MUST never be used in a production setup.
```

En allant sur <http://127.0.0.1:8000/event/> on tombe sur notre magnifique page :



Eventify

+ Créer un événement

Create new Event

Titre de l'événement

Description

Date de l'événement

Nombre de places

Lieu

Enregistrer

[back to list](#)

200 @app_event_new 3416 ms 34.0 MIB 1 1 9 n/a 497 ms

Server v8.1.6

Eventify

+ Créer un événement

Événements à venir

Rechercher

Réinitialiser

Concert de Rock
Groupe les rockeurs...

Date : 28/04/2027 12:00

Lieu : Oraison

Places : 150

Voir plus

Modifier

Laser Game
avec pleins de lasers...

Date : 22/02/2222 22:22

Lieu : Paris

Places : 2000

Voir plus

Modifier

200 @app_event_index 529 ms 30.0 MIB 1 1 n/a 14 ms 1 in 0.88 ms

Server v8.1.6

6) Conclusion

En conclusion j'ai compris l'intérêt que l'on peut avoir à utiliser Symfony, il permet de simplifier beaucoup d'actions, la gestion du front end et du backend est beaucoup plus simple à faire grâce à cet outil.